MySQL String + Result Functions

These are practical functions used often in cleaning data, formatting reports, and querying smartly.

---

1. CHAR_LENGTH() in MySQL

Purpose:

Returns **number of characters** in a string.

SELECT CHAR_LENGTH('Indra');

Output:

5

Important Difference: LENGTH() vs CHAR_LENGTH()

SELECT LENGTH('A');

SELECT CHAR_LENGTH('A');

Both = 1 for English.

But for Unicode/multibyte chars:

SELECT LENGTH('₹');

SELECT CHAR_LENGTH('₹');

- LENGTH() = bytes used in storage
- CHAR_LENGTH() = actual visible characters

Use Case:

Check username length.

SELECT fname, CHAR_LENGTH(fname)

FROM employee;

---

2. LEFT()

Purpose:

Returns left-side characters from a string.

Syntax:

LEFT(string, number)

Example:

SELECT LEFT('MYSQL', 3);

Output:

MYS

Use Case:

Get prefix / first letters.

SELECT LEFT(fname,2)

FROM employee;

---

3. RIGHT()

Purpose:

Returns right-side characters.

SELECT RIGHT('MYSQL',2);

Output:

QL

Use Case:

Last digits, suffix, domain endings.

SELECT RIGHT('9876543210',4);

Output:

3210

---

4. TRIM()

Purpose:

Removes unwanted spaces (or chosen chars) from start/end.

Basic Example

SELECT TRIM(' Indra ');

Output:

Indra

Remove Specific Character

SELECT TRIM('*' FROM '***hello***');

Output:

hello

Also Exists:

LTRIM() -- left spaces only

RTRIM() -- right spaces only

Real Use Case:

Messy imported CSV data.

SELECT TRIM(name)

FROM customers;

---

5. INSERT() Function

Purpose:

Replace part of string starting from position.

Syntax:

INSERT(str, pos, len, new_string)

Means:

- start at pos
- remove len chars
- insert new string

Example:

SELECT INSERT('Indra sena',6,2,'Hi sir');

Let's break:

Indra sena

Position 6 starts at space / s depending counting:

I n d r a _ s e n a
1 2 3 4 5 6 7 8 9 10

Remove 2 chars from pos 6 → ' s'

Insert 'Hi sir'

Output similar:

IndraHi sirena

(Depends exact spacing)

Cleaner Example:

SELECT INSERT('abcdef',2,3,'XYZ');

Output:

aXYZef

Use Case:

Mask / replace middle data.

---

6. REPEAT()

Purpose:

Repeat a string multiple times.

SELECT REPEAT('Hi',3);

Output:

HiHiHi

Use Case:

Create visual separators

SELECT REPEAT('-',20);

Output:

Mask password

SELECT REPEAT('*',8);

Output:

****

---

7. DISTINCT

Purpose:

Returns unique values only.

This is not a string function—it is a **SELECT modifier**.

Example:

Table:

dept

Loan

Cash

Loan

Deposit

SELECT DISTINCT dept FROM employee;

Output:

dept

Loan

Cash

Deposit

Multiple Columns

SELECT DISTINCT dept, desig

FROM employee;

Unique combinations returned.

Internal Logic of DISTINCT

MySQL compares selected columns and removes duplicates using sorting/hash strategy.

Can be slower on huge data if no indexes.

Real Use Cases

Get all departments

SELECT DISTINCT dept FROM employee;

Unique cities

SELECT DISTINCT city FROM customers;

Practice on Your Employee Table

SELECT CHAR_LENGTH(fname) FROM employee;

SELECT LEFT(fname,2) FROM employee;

SELECT RIGHT(lname,3) FROM employee;

SELECT TRIM(' hello ');

SELECT INSERT(fname,2,2,'XX') FROM employee;

SELECT REPEAT('*',5);

SELECT DISTINCT dept FROM employee;

Golden Rule

These functions mostly change display/output only.

Unless used in:

UPDATE table_name

SET fname = TRIM(fname);

Then real data changes.

Common Beginner Mistakes

Wrong:

SELECT LEFT(fname);

Need count:

LEFT(fname,2)

Wrong:

SELECT DISTINCT(dept)

Works sometimes, but cleaner:

SELECT DISTINCT dept

Pro Interview Insight

Use DISTINCT carefully. Many beginners use it to hide duplicate problems caused by bad joins.

Fix joins first. Use DISTINCT second.

Summary Table

Function	Purpose
CHAR_LENGTH()	Count characters
LEFT()	Left side chars
RIGHT()	Right side chars
TRIM()	Remove spaces/chars
INSERT()	Replace portion of string
REPEAT()	Repeat string
DISTINCT	Unique rows

DISTINCT in MySQL

DISTINCT looks simple, but many people misuse it. Real SQL users understand **when to use it, when not to use it, and what it costs**.

---

1. Simple Explanation (Beginner Level)

DISTINCT removes duplicate rows from the result set.

Syntax

SELECT DISTINCT column_name

FROM table_name;

Example

Table employee

emp_id	fname	dept
101	Raju	Loan
102	Sham	Cash
103	Baburao	Loan
104	Paul	Account

Query:

SELECT DISTINCT dept

FROM employee;

Output:

dept
Loan
Cash
Account

Because Loan repeated twice, shown once.

---

2. Deep Explanation (How It Works Internally)

Important Truth:

DISTINCT works **after rows are selected**, on the columns in the SELECT list.

It checks duplicate combinations and keeps one unique row.

Logical Query Execution Order

When SQL runs:

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT

6. DISTINCT
7. ORDER BY
8. LIMIT

So DISTINCT happens **after SELECT values are formed**.

Internal Engine Behavior

MySQL may use:

A) Sorting method

Sort rows, then remove adjacent duplicates.

B) Hashing / temporary table method

Store seen values and eliminate repeats.

Depends on query plan, indexes, row size.

---

3. Real-World Use Cases (How Companies Use It)

A. Unique customers who purchased today

```
SELECT DISTINCT customer_id
FROM orders
WHERE order_date = CURDATE();
```

B. Unique cities users signed up from

```
SELECT DISTINCT city
FROM users;
```

C. Unique device types in logs

```
SELECT DISTINCT device_type
FROM app_events;
```

D. Analytics dashboards

Populate dropdown filters:

- departments
- countries

- categories

---

4. Practice Problems (Easy → Hard)

Use table employee(emp_id, fname, dept, desig)

Q1 Easy

Show unique departments.

Q2 Easy

Show unique designations.

Q3 Medium

Show unique department + designation combinations.

Q4 Medium

Show unique first names only.

Q5 Hard

How many unique departments exist?

---

5. Solutions

```
-- Q1
SELECT DISTINCT dept FROM employee;

-- Q2
SELECT DISTINCT desig FROM employee;

-- Q3
SELECT DISTINCT dept, desig FROM employee;

-- Q4
SELECT DISTINCT fname FROM employee;

-- Q5
SELECT COUNT(DISTINCT dept) FROM employee;
```

---

6. Very Important Concept: Multi-Column DISTINCT

Query:

```
SELECT DISTINCT dept, desig
FROM employee;
```

This removes duplicate **pairs**, not individual columns.

Example:

dept	desig
Loan	Manager
Loan	Associate
Loan	Manager

Result:

dept	desig
Loan	Manager
Loan	Associate

---

7. Common Mistakes

Mistake 1: Using DISTINCT to hide bad JOINs

```
SELECT DISTINCT c.name
FROM customers c
JOIN orders o ON c.id = o.customer_id;
```

This may hide duplicate rows caused by multiple orders.

Better logic may be:

```
SELECT c.name
FROM customers c
WHERE EXISTS (
  SELECT 1
  FROM orders o
  WHERE o.customer_id = c.id
);
```

Mistake 2: Thinking DISTINCT sorts rows

Wrong.

```
SELECT DISTINCT dept FROM employee;
```

Output order is not guaranteed.

Use:

```
SELECT DISTINCT dept
FROM employee
ORDER BY dept;
```

Mistake 3: DISTINCT on primary key

```
SELECT DISTINCT emp_id FROM employee;
```

Wasteful if emp_id already unique.

---

8. Performance Awareness

Expensive on Large Tables Because:

- sorting large result sets
- temp tables
- memory/disk usage

Better with indexes:

If column indexed:

```
SELECT DISTINCT dept FROM employee;
```

Can be faster.

---

9. DISTINCT vs GROUP BY

These often produce same result:

```
SELECT DISTINCT dept FROM employee;
SELECT dept FROM employee GROUP BY dept;
```

But intent differs:

- DISTINCT = remove duplicates
- GROUP BY = aggregate groups

Use right tool.

---

10. Edge Cases

NULL Handling

Multiple NULLs treated as duplicates.

Table:

dept
Loan
NULL
NULL

SELECT DISTINCT dept FROM employee;

Result:

dept
Loan
NULL

Only one NULL shown.

Spaces / Case

Depending collation:

Loan, loan, LOAN may be treated same or different.

---

11. Twist Problem (Think Like Interviewer)

Table:

id	name
1	Raj
2	Raj
3	raj

Query:

SELECT DISTINCT name FROM users;

How many rows returned?

Answer:

Depends on collation (case-sensitive or not).

Strong SQL users know behavior depends on schema settings.

---

12. Best Practices

Good:

SELECT DISTINCT country FROM customers;

Bad:

SELECT DISTINCT * FROM huge_table;

Expensive and often meaningless.

---

13. Mental Model

Think:

DISTINCT compares the final selected columns and keeps one copy of each unique combination.

---

14. Interview Challenge

Given:

SELECT DISTINCT dept

FROM employee

ORDER BY fname;

Will this always work?

Answer:

In some SQL modes/databases problematic because fname not selected. MySQL may allow depending mode/version, but conceptually risky.

ORDER BY in MySQL

ORDER BY is simple on surface, but real SQL users understand **sorting logic, execution order, NULL behavior, indexes, and performance costs**.

---

1. Simple Explanation (Beginner Level)

ORDER BY sorts rows in ascending or descending order.

Syntax

SELECT column1, column2

FROM table_name

ORDER BY column_name;

Default = ascending (ASC)

SELECT * FROM employee

ORDER BY fname;

Descending

SELECT * FROM employee

ORDER BY fname DESC;

Example

Table:

emp_id	fname	salary
103	Ravi	50000
101	Amit	70000
102	Zara	60000

SELECT * FROM employee

ORDER BY fname;

Output:

Amit, Ravi, Zara

---

2. Deep Explanation (How It Works Internally)

Logical Query Execution Order

SQL generally processes:

1. FROM

2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. DISTINCT
7. ORDER BY
8. LIMIT

So sorting happens **near the end**, after rows are selected.

Internal Engine Behavior

To sort rows, MySQL may:

A) Use Index Order (Fast)

If sorting column has index.

```
SELECT * FROM employee
ORDER BY emp_id;
```

If emp_id indexed, MySQL may read already sorted rows.

B) Filesort (Extra Work)

If no suitable index exists, MySQL sorts separately.

This can use:

- memory
- temporary space
- disk (large data)

---

3. Real-World Use Cases (How Companies Use It)

A. Latest orders first

```
SELECT *
FROM orders
ORDER BY created_at DESC;
```

B. Top paid employees

```
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 10;
```

C. Alphabetical customer list

```
SELECT name
FROM customers
ORDER BY name;
```

D. Dashboard ranking

```
SELECT product_name, revenue
FROM sales
ORDER BY revenue DESC;
```

---

4. Practice Problems (Easy → Hard)

Use employee(emp_id, fname, dept, salary)

Q1 Easy

Sort by first name A-Z.

Q2 Easy

Sort by salary highest first.

Q3 Medium

Sort by department then name.

Q4 Medium

Show top 3 highest salaries.

Q5 Hard

Sort by salary descending, and if same salary sort by fname ascending.

---

5. Solutions

```
-- Q1
SELECT * FROM employee ORDER BY fname;
```

-- Q2

SELECT * FROM employee ORDER BY salary DESC;

-- Q3

SELECT * FROM employee ORDER BY dept, fname;

-- Q4

SELECT * FROM employee ORDER BY salary DESC LIMIT 3;

-- Q5

SELECT * FROM employee ORDER BY salary DESC, fname ASC;

---

6. Multi-Column Sorting

SELECT *

FROM employee

ORDER BY dept ASC, salary DESC;

Meaning:

1. Sort by dept
2. Inside each dept, sort by salary high to low

---

7. Common Mistakes

Mistake 1: Assuming default order exists

Wrong:

SELECT * FROM employee;

No guaranteed order.

Always use ORDER BY if order matters.

Mistake 2: LIMIT without ORDER BY

SELECT * FROM employee LIMIT 5;

Returns arbitrary 5 rows.

Use:

```
SELECT * FROM employee
ORDER BY created_at DESC
LIMIT 5;
```

Mistake 3: Sorting text numbers

If salary stored as VARCHAR:

salary
100
20
9

Sort gives:

100, 20, 9 (string order)

Fix schema or cast.

---

8. Performance Awareness

Expensive Sort When:

- huge table
- no index
- sorting text columns
- sorting many rows then LIMIT late

Faster Query

```
SELECT emp_id, fname
FROM employee
ORDER BY emp_id;
```

If emp_id indexed.

Useful Index Example

INDEX(created_at)

Helps:

```
SELECT *
FROM orders
ORDER BY created_at DESC
LIMIT 20;
```

Very common production pattern.

---

9. ORDER BY with Aliases

```
SELECT fname, salary * 12 AS annual_salary
FROM employee
ORDER BY annual_salary DESC;
```

Allowed.

---

10. ORDER BY by Position (Not Recommended)

```
SELECT fname, salary
FROM employee
ORDER BY 2 DESC;
```

Means second selected column.

Works, but bad readability.

---

11. NULL Behavior

MySQL commonly sorts NULL first in ASC, last in DESC.

```
SELECT * FROM employee
ORDER BY bonus ASC;
```

Rows with NULL bonus may appear first.

---

12. Twist Problem (Interviewer Style)

```
SELECT dept, COUNT(*)
FROM employee
GROUP BY dept
```

ORDER BY COUNT(*) DESC;

Valid?

Yes.

Sorts grouped result by employee count.

---

13. Real Engineer Thinking

Need newest records?

ORDER BY created_at DESC LIMIT 1

Need top customers?

ORDER BY revenue DESC LIMIT 10

Need stable pagination?

ORDER BY id LIMIT 20 OFFSET 40

---

14. Best Practices

Good

SELECT emp_id, fname

FROM employee

ORDER BY emp_id;

Better than:

SELECT *

FROM employee

ORDER BY emp_id;

Avoid SELECT * when unnecessary.

LIKE in MySQL

LIKE is one of the most used filters in SQL. Beginners use it for simple search. Strong SQL users understand **pattern matching, wildcard logic, performance impact, collation behavior, and safe querying**.

---

1. Simple Explanation (Beginner Level)

LIKE is used in WHERE to match text patterns.

Syntax

```
SELECT *
FROM table_name
WHERE column_name LIKE 'pattern';
```

Wildcards

% = any number of characters (0 or more)

```
SELECT *
FROM employee
WHERE fname LIKE 'R%';
```

Starts with R

Examples matched:

- Raju
- Ravi
- Rahul

_ = exactly one character

```
SELECT *
FROM employee
WHERE fname LIKE 'R___';
```

Matches 4-letter names starting with R.

Example:

- Ravi

---

2. Deep Explanation (How It Works Internally)

Query Execution Order

```
SELECT ...
FROM ...
WHERE fname LIKE 'R%'
```

SQL processes:

1. FROM
2. WHERE (LIKE filter applied here)
3. SELECT
4. ORDER BY
5. LIMIT

So LIKE filters rows **before output**.

How Matching Works

MySQL compares each row's string against pattern.

Example:

fname LIKE '%vi%'

Checks if substring vi exists anywhere.

Collation Matters

Many MySQL collations are case-insensitive.

So:

WHERE fname LIKE 'raj%'

May match:

- Raj
- RAJ
- raj

Depends on table/database collation.

---

3. Real-World Use Cases (How Companies Use It)

A. Search users by prefix

SELECT *

FROM users

WHERE name LIKE 'Ind%';

Autocomplete starts-with search.

B. Email domain filtering

```
SELECT *
FROM customers
WHERE email LIKE '%@gmail.com';
```

C. Product keyword search

```
SELECT *
FROM products
WHERE product_name LIKE '%phone%';
```

D. Detect bad data

```
SELECT *
FROM contacts
WHERE phone LIKE '____';
```

Exactly 4 characters.

---

4. Practice Problems (Easy → Hard)

Use employee(emp_id, fname, lname, dept)

Q1 Easy

Find names starting with P

Q2 Easy

Find names ending with a

Q3 Medium

Find names containing ra

Q4 Medium

Find names with exactly 4 letters

Q5 Hard

Find employees whose lname starts with M and dept contains ash

---

5. Solutions

```
-- Q1
```

```
SELECT * FROM employee WHERE fname LIKE 'P%';

-- Q2
SELECT * FROM employee WHERE fname LIKE '%a';

-- Q3
SELECT * FROM employee WHERE fname LIKE '%ra%';

-- Q4
SELECT * FROM employee WHERE fname LIKE '____';

-- Q5
SELECT * FROM employee
WHERE lname LIKE 'M%'
AND dept LIKE '%ash%';
```

---

6. Common Patterns You Must Know

Starts with

LIKE 'A%'

Ends with

LIKE '%n'

Contains

LIKE '%abc%'

Exact 5 chars

LIKE '_____'

Starts A ends n

LIKE 'A%n'

---

7. Common Mistakes

Mistake 1: Using = instead of LIKE

Wrong:

WHERE fname = '%Raj%'

Correct:

WHERE fname LIKE '%Raj%'

Mistake 2: _ vs %

LIKE '_'

Means exactly 1 character, not many.

Mistake 3: Leading wildcard kills index usage

WHERE fname LIKE '%raj'

Often slower than:

WHERE fname LIKE 'raj%'

Because prefix searches can use indexes more easily.

---

8. Performance Awareness

Fast(er):

WHERE fname LIKE 'Ra%'

May use index on fname.

Slow(er):

WHERE fname LIKE '%aj%'

Usually scans many rows.

Real Company Tip

For advanced search at scale:

- Full-text indexes
- Search engines (Elasticsearch/OpenSearch)

Not endless %keyword% queries.

---

9. LIKE with Numbers (Possible but Risky)

WHERE phone LIKE '98%'

Works if stored as text.

If numeric column, design issue usually.

---

10. Escaping Special Characters

Suppose searching actual %

WHERE remarks LIKE '%\%%'

Need escaping depending SQL mode.

---

11. LIKE + ORDER BY + LIMIT

Very common:

SELECT *

FROM users

WHERE name LIKE 'Ra%'

ORDER BY name

LIMIT 10;

Autocomplete search.

---

12. Twist Problem (Interviewer Style)

Which is faster on indexed fname?

A)

LIKE 'Ra%'

B)

LIKE '%Ra%'

Answer:

A is usually much faster.

---

13. Edge Cases

Empty String

LIKE '%'

Matches all non-NULL strings including empty string.

NULL Values

fname LIKE 'R%'

NULL rows do not match.

Need:

WHERE fname IS NULL

---

14. Best Practices

Good

WHERE email LIKE '%@gmail.com'

Better Search Logic

WHERE LOWER(name) LIKE 'raj%'

If case-sensitive collation and you need case-insensitive matching (watch performance).